

Explanation-Oriented Design

Andrew Raybould

... in which I oppose the practice of leaving the understanding of a design as an exercise for the reader.

When computer-aided software engineering became “the next big thing” for software development in the '90s, I really believed it would make a difference, that it could cut through the fog of confusion and lead to designs of unprecedented clarity.

Within five years, though, CASE was gone, sunk by contentious disputes over methods, disillusion induced by its failure to live up to unrealistic claims, and its slow response to the arrival of object orientation.

Beyond these reasons, however, I felt that something more fundamental was missing. These tools and methods produced designs that simply weren't

particularly useful for developing systems. Understanding the designers' intent, and even answering the basic question, “How does it work?” required a sort of mental reverse engineering. Object orientation's advent has, contrary to expectations, produced only a modest clearing of the view.

The need for explanation

Combine this with the observation that a project's success depends critically on the depth and breadth of the team's understanding of the task, and it's clear how our approach to design must change to move beyond these problems. All but the simplest designs must be

actively explained to those who need to understand them, not just presented as if they were self-evident facts. Designs must be taught, and their documentation must act as tutorials, not just descriptive references.

The design process generates a large amount of explanatory information, in the form of arguments for correctness, efficiency, and general fitness, and in the reasons for rejecting alternatives. At a more abstract level, a designer doesn't introduce an element into a design without having some purpose for it, and knowing its purpose is often the key to understanding it.

The cost of letting this hard-won information slip away is visible everywhere, and not just in the effort to regenerate it that's expended by anyone who needs any part of it. I often see teams run in circles because they've forgotten key findings, and it's not unusual for everyone leaving a meeting to have a different idea of what was decided. Poor understanding is a drag on the process, and worse: it leads to poor decisions, confusion, and, ultimately, chaos.

Cascading benefits

Conversely, writing an explanation of what you're doing, as you go, brings cascading benefits. Most obviously, you can get everyone on the same page. More important, perhaps, is that doing so is an excellent way to separate what you really understand from what you

Continued on p. 103



Continued from p. 104

just think you know, and what you never realized you don't know. I often find better, and usually simpler, ways of doing things: the first person to learn from an explanation-oriented design is its author.

We're all familiar with explanatory technical writing of various styles: articles and papers in technical journals, lecture notes, presentations, textbook chapters, and even the description section of patents. An explanation-oriented design grows into a hierarchy of topics, each described this way. Ideally, the work is threaded together with hypertext links and indexed by a search engine.

The problem with pictures

These examples point to my one area of contention with the established view of how to document designs, which is through graphical models. From an explanation-oriented viewpoint, diagrams have a serious problem: they lack the expressive power with which to explain rational arguments, especially about abstract concepts (if you disagree, draw a diagram that, by itself, presents a counterargument).

Explanatory technical writing, in contrast, uses text as the primary mode of expression, with diagrams playing an important, but subsidiary, role of illustrating selected aspects, particularly relationships between things. This is also typical of explanation-oriented designs.

Agility and formality

It's important to realize that agile methods don't do away with design; they perform it by other means. Agile teams depend greatly on their members' understanding of those designs, and so will benefit from clear, pertinent explanations. For greater agility, focus mainly on explaining your design's most difficult, critical, and counter-intuitive aspects. At the other extreme, for mission-critical systems where reliability is of the essence, you might want your explanations to include rigorous formal proofs of correctness and performance.

Copyright and reprint permission: Copyright © 2006 by the Institute of Electrical and Electronics Engineers, Inc. All rights reserved. Abstracting is permitted with credit to the source. Libraries are permitted to photocopy beyond the limits of US copyright law for private use of patrons those post-1977 articles that carry a code at the bottom of the first page, provided the per-copy fee indicated in the code is paid through the Copyright Clearance Center, 222 Rosewood Dr., Danvers, MA 01923. For copying, reprint, or republication permission, write to Copyright and Permissions Dept., IEEE Publications Admin., 445 Hoes Ln., Piscataway, NJ 08855-1331.

IEEE Software (ISSN 0740-7459) is published bimonthly by the IEEE Computer Society. IEEE headquarters: Three Park Ave., 17th Floor, New York, NY 10016-5997. IEEE Computer Society Publications Office: 10662 Los Vaqueros Cir., PO Box 3014, Los Alamitos, CA 90720-1314; +1 714 821 8380; fax +1 714 821 4010. IEEE Computer Society headquarters: 1730 Massachusetts Ave. NW, Washington, DC 20036-1903. Subscription rates: IEEE Computer Society members get the lowest rate of US\$47 per year, which includes printed issues plus online access to all issues published since 1988. Go to www.computer.org/subscribe to order and for more information on other subscription prices. Back issues: \$20 for members, \$128 for nonmembers (plus shipping and handling). This magazine is available on microfiche.

Postmaster: Send undelivered copies and address changes to *IEEE Software*, Membership Processing Dept., IEEE Service Center, 445 Hoes Lane, Piscataway, NJ 08855-1331. Periodicals Postage Paid at New York, NY, and at additional mailing offices. Canadian GST #125634188. Canada Post Publications Mail Agreement Number 40013885. Return undeliverable Canadian addresses to PO Box 122, Niagara Falls, ON L2E 6S8, Canada. Printed in the USA.

This idea's roots go back at least to Donald Knuth's book *Literate Programming* from 1973,¹ but it has never entered the mainstream. One reason might lie, paradoxically, in some of its strengths: demanding neither new tools nor special knowledge, it enhances any methodology and can be adopted incrementally. Unfortunately, people are more excited by radical ideas, especially when "validated" by the existence of tools and certification

programs. You can choose, however, to advance this principle by asking one simple question: how can I help my colleagues understand my design? ☞

Reference

1. D. Knuth, *Literate Programming*, Center for the Study of Language and Information, Stanford Univ., 1992.

Andrew Raybould develops trading systems for the Securities Industry Automation Corp. Contact him at andrew.raybould@ieee.org.

Stay on Track

IEEE Internet Computing reports emerging tools, technologies, and applications implemented through the Internet to support a worldwide computing environment.

In 2006-7, we'll look at

- Web Services for Geographic Information Systems
- Malicious Software
- Autonomic Computing

... and more!

IEEE Internet Computing

www.computer.org/internet

